

This is a sample Jupyter Notebook

Below is an example of a code cell. Put your cursor into the cell and press Shift+Enter to execute it and select the next one, or click 'Run Cell' button.

Press Double Shift to search everywhere for classes, files, tool windows, actions, and settings.

To learn more about Jupyter Notebooks in PyCharm, see [help](#). For an overview of PyCharm, go to Help -> Learn IDE features or refer to [our documentation](#).

```
In [9]: from doctest import DocFileCase

#importing visualisation Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#importing metrics
from sklearn.metrics import accuracy_score, classification_report, auc, roc_curve
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV

#importing preprocessing
from category_encoders import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

#importing regression models
from sklearn.feature_selection import f_regression

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVR
```

```
In [10]: #Loading the dataset
data = pd.read_csv('data/Myrtaceae-Fruit-Flies-&-T-leucotreta-V3.csv')
data.head()
```

Out[10]:

	Sample no.	Plant family	Plant species	Plant Name	Exotic/Indigenous	Locality	Latitude	Long
0	1	Myrtaceae	Acca sellowiana (O.Berg) Burret	Feijoa	Exotic	ARC-TSC Nelspruit	-25.452592	30.97
1	2	Myrtaceae	Acca sellowiana (O.Berg) Burret	Feijoa	Exotic	ARC-TSC Nelspruit	-25.452592	30.97
2	3	Myrtaceae	Acca sellowiana (O.Berg) Burret	Feijoa	Exotic	ARC-TSC Nelspruit	-25.452592	30.97
3	4	Myrtaceae	Acca sellowiana (O.Berg) Burret	Feijoa	Exotic	ARC-TSC Nelspruit	-25.452592	30.97
4	5	Myrtaceae	Acca sellowiana (O.Berg) Burret	Feijoa	Exotic	ARC-TSC Nelspruit	-25.452592	30.97

Exploration and Data Cleaning

```
In [11]: data.drop_duplicates(keep='first', inplace=True)
data.describe().T
```

Out[11]:

	count	mean	std	min	25%	50%	75
Sample no.	204.0	11.808824	8.903850	1.000000	5.000000	10.000000	16.0000
Latitude	204.0	-24.962784	0.751675	-26.512358	-25.452592	-25.452592	-23.8596
Longitude	204.0	30.792689	0.327001	29.985615	30.298042	30.971381	30.9713
Altitude (m)	203.0	673.935961	129.421680	448.000000	670.000000	670.000000	670.0000
No. fruit	204.0	71.166667	118.405921	3.000000	24.000000	40.000000	73.2500
Weight (gram)	204.0	598.375980	874.790333	5.500000	209.850000	368.450000	624.3500
C-rosa-sensu-lato-count	165.0	40.836364	60.012731	0.000000	0.000000	10.000000	64.0000

In [12]: `data = data.dropna(how='all', axis=1)`
`data.describe().T`

Out[12]:

	count	mean	std	min	25%	50%	75
Sample no.	204.0	11.808824	8.903850	1.000000	5.000000	10.000000	16.0000
Latitude	204.0	-24.962784	0.751675	-26.512358	-25.452592	-25.452592	-23.8596
Longitude	204.0	30.792689	0.327001	29.985615	30.298042	30.971381	30.9713
Altitude (m)	203.0	673.935961	129.421680	448.000000	670.000000	670.000000	670.0000
No. fruit	204.0	71.166667	118.405921	3.000000	24.000000	40.000000	73.2500
Weight (gram)	204.0	598.375980	874.790333	5.500000	209.850000	368.450000	624.3500
C-rosa-sensu-lato-count	165.0	40.836364	60.012731	0.000000	0.000000	10.000000	64.0000

In [13]: `print(data.isnull().sum())`

Sample no.	0
Plant family	0
Plant species	0
Plant Name	0
Exotic/Indigenous	13
Locality	0
Latitude	0
Longitude	0
Altitude (m)	1
Köppen-Geiger climate classification	0
Local Municipality	0
District Municipality	0
Province	0
Tree/ground	0
Maturity stage	0
Date sampled	0
No. fruit	0
Weight (gram)	0
C-rosa-sensu-lato-count	39
dtype:	int64

```
In [14]: mode = data['Exotic/Indigenous'].mode()
data['Exotic/Indigenous'].fillna('Exotic', inplace=True)

mean = data['Altitude (m) '].mean()
data['Altitude (m) '].fillna(mean, inplace=True)
print(data.isnull().sum())
```

Sample no.	0
Plant family	0
Plant species	0
Plant Name	0
Exotic/Indigenous	0
Locality	0
Latitude	0
Longitude	0
Altitude (m)	0
Köppen-Geiger climate classification	0
Local Municipality	0
District Municipality	0
Province	0
Tree/ground	0
Maturity stage	0
Date sampled	0
No. fruit	0
Weight (gram)	0
C-rosa-sensu-lato-count	39
dtype:	int64

C:\Users\TokeloQhena\AppData\Local\Temp\ipykernel_38608\3299062901.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
data['Exotic/Indigenous'].fillna('Exotic', inplace=True)
```

C:\Users\TokeloQhena\AppData\Local\Temp\ipykernel_38608\3299062901.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
data['Altitude (m) '].fillna(mean, inplace=True)
```

```
In [15]: monthSampled = pd.to_datetime(data['Date sampled'])
         data['Month Sampled'] = monthSampled.dt.month
```

C:\Users\TokeloQhena\AppData\Local\Temp\ipykernel_38608\2053300272.py:1: UserWarning: Parsing dates in %d/%m/%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.

```
monthSampled = pd.to_datetime(data['Date sampled'])
```

Data Visualization

```
In [16]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 20 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Sample no.                               204 non-null    int64
1   Plant family                             204 non-null    object
2   Plant species                             204 non-null    object
3   Plant Name                               204 non-null    object
4   Exotic/Indigenous                         204 non-null    object
5   Locality                                 204 non-null    object
6   Latitude                                 204 non-null    float64
7   Longitude                                204 non-null    float64
8   Altitude (m)                             204 non-null    float64
9   Köppen-Geiger climate classification      204 non-null    object
10  Local Municipality                       204 non-null    object
11  District Municipality                   204 non-null    object
12  Province                                 204 non-null    object
13  Tree/ground                             204 non-null    object
14  Maturity stage                           204 non-null    object
15  Date sampled                             204 non-null    object
16  No. fruit                                204 non-null    int64
17  Weight (gram)                            204 non-null    float64
18  C-rosa-sensu-lato-count                  165 non-null    float64
19  Month Sampled                            204 non-null    int32
dtypes: float64(5), int32(1), int64(2), object(12)
memory usage: 31.2+ KB
```

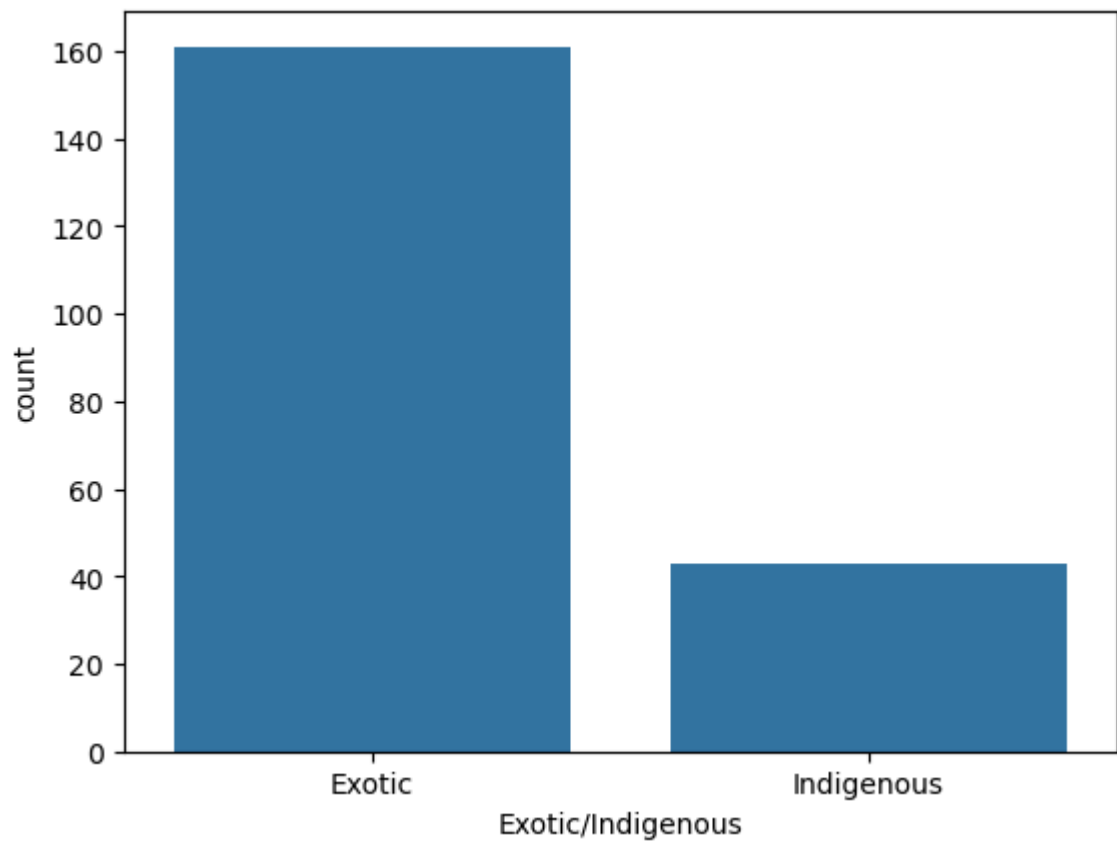
```
In [17]: columns_to_delete = ['Sample no.', 'Plant species', 'Plant Name', 'Local Municipality']

for column in columns_to_delete:
    data = data.drop(data.filter(like=column,axis=1).columns.to_list(), axis=1)

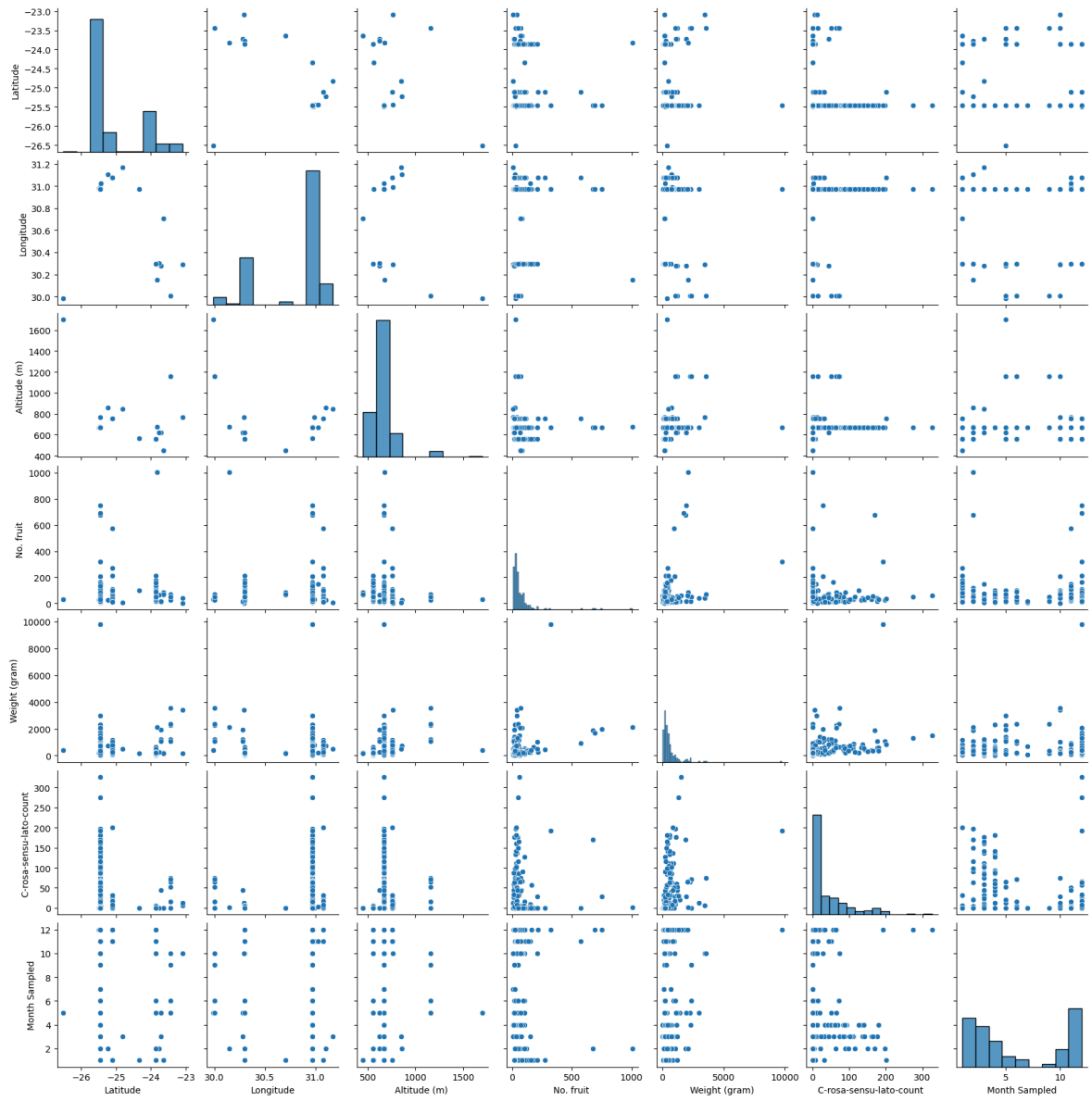
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 13 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Plant family                             204 non-null    object
1   Exotic/Indigenous                         204 non-null    object
2   Latitude                                 204 non-null    float64
3   Longitude                                204 non-null    float64
4   Altitude (m)                             204 non-null    float64
5   Köppen-Geiger climate classification      204 non-null    object
6   Tree/ground                             204 non-null    object
7   Maturity stage                           204 non-null    object
8   Date sampled                             204 non-null    object
9   No. fruit                                204 non-null    int64
10  Weight (gram)                            204 non-null    float64
11  C-rosa-sensu-lato-count                  165 non-null    float64
12  Month Sampled                            204 non-null    int32
dtypes: float64(5), int32(1), int64(1), object(6)
memory usage: 20.1+ KB
```

```
In [18]: sns.countplot(data=data, x='Exotic/Indigenous')
plt.show()
```



```
In [19]: sns.pairplot(data)  
plt.show()
```



Encoding

```
In [20]: columns_to_encode = ['Exotic/Indigenous', 'Köppen-Geiger climate classification',

df_encoded_data = pd.DataFrame()

for column in columns_to_encode:

    df_column_encoding = OneHotEncoder(use_cat_names=True).fit_transform(data, fi

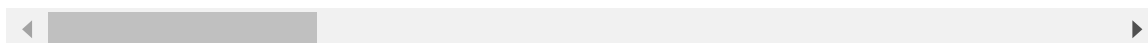
    df_encoded_data = df_encoded_data._append(df_column_encoding)

df_encoded_data.head()
```


Out[20]:

	Plant family_Myrtaceae	Plant family_Myertacea	Exotic/Indigenous_Exotic	Exotic/Indigenous_Indi
0	1	0	1	
1	1	0	1	
2	1	0	1	
3	1	0	1	
4	1	0	1	

5 rows × 133 columns



Feature and target split

```
In [22]: target_column = 'C-rosa-sensu-lato-count'

feature_data = df_encoded_data.drop(df_encoded_data.filter(like=target_column,axis=1))

target = df_encoded_data.filter(like=target_column,axis=1)
target.fillna(0, inplace=True)

feature_data.head()
```

C:\Users\TokeloQhena\AppData\Local\Temp\ipykernel_38608\2087823322.py:6: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

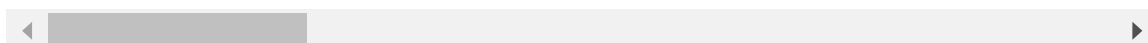
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
target.fillna(0, inplace=True)
```

Out[22]:

	Plant family_Myrtaceae	Plant family_Myertacea	Exotic/Indigenous_Exotic	Exotic/Indigenous_Indi
0	1	0	1	
1	1	0	1	
2	1	0	1	
3	1	0	1	
4	1	0	1	

5 rows × 132 columns



In [23]: target.head()

Out[23]: **C-rosa-sensu-lato-count**

0	197.0
1	59.0
2	25.0
3	62.0
4	57.0

In [24]: `print(feature_data.isnull().sum())`

```
Plant family_Myrtaceae      0
Plant family_Myertacea      0
Exotic/Indigenous_Exotic    0
Exotic/Indigenous_Indigenous 0
Latitude                    0
..
Date sampled_07/02/2016     0
Date sampled_01/03/2016     0
No. fruit                   0
Weight (gram)               0
Month Sampled               0
Length: 132, dtype: int64
```

Split Traning and Test Data

In [25]: `x_train, x_test, y_train, y_test = train_test_split(feature_data,target, test_si
y_train.head()`

Out[25]: **C-rosa-sensu-lato-count**

111	61.0
158	7.0
95	24.0
115	72.0
25	13.0

Scaling

In [26]: `#feature scaling`

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

Models

```
In [27]: lr = LinearRegression()
lr.fit(x_train,y_train)
print(f"Training Accuracy Score :{lr.score(x_train, y_train) * 100:.1f}%")
print(f"Validation Accuracy Score :{lr.score(x_test, y_test) * 100:.1f}%")
```

Training Accuracy Score :72.5%
Validation Accuracy Score :67.5%

```
In [28]: dt= DecisionTreeRegressor()
dt.fit(x_train,y_train)
print(f"Training Accuracy Score :{dt.score(x_train, y_train) * 100:.2f}%")
print(f"Validation Accuracy Score :{dt.score(x_test, y_test) * 100:.2f}%")
```

Training Accuracy Score :100.00%
Validation Accuracy Score :100.00%

```
In [29]: rf = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=1)
rf.fit(x_train,y_train)
print(f"Training Accuracy Score :{rf.score(x_train, y_train) * 100:.2f}%")
print(f"Validation Accuracy Score :{rf.score(x_test, y_test) * 100:.2f}%")
```

c:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:1473: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
return fit_method(estimator, *args, **kwargs)

Training Accuracy Score :74.42%
Validation Accuracy Score :73.03%

```
In [30]: gnb = GaussianNB()
gnb.fit(x_train,y_train)
print(f"Training Accuracy Score :{gnb.score(x_train, y_train) * 100:.2f}%")
print(f"Validation Accuracy Score :{gnb.score(x_test, y_test) * 100:.2f}%")
```

c:\ProgramData\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1339: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
y = column_or_1d(y, warn=True)

Training Accuracy Score :89.46%
Validation Accuracy Score :85.78%